Missão Prática | Nível 4 | Mundo 3

Marcelo Picado Schulze | 202403781506

Campus BANCÁRIOS
Nível 4: Vamos integrar sistemas – 202403781506 – 3° periodo - semestre 2025.1

Objetivo da Prática

1. Implementar persistência com base em JPA.
2. Implementar regras de negócio na plataforma JEE, através de EJBs.
3. Implementar sistema cadastral Web com base em Servlets e JSPs.
4. Utilizar a biblioteca Bootstrap para melhoria do design.
5. No final do exercício, o aluno terá criado todos os elementos necessários para exibição e entrada de dados na plataforma Java Web, tornando-se capacitado para lidar com contextos reais de aplicação.

1º Procedimento | Camadas de Persistência e Controle

```
<persistence xmlns="http://xmlns.jcp.org/xml/ns/persistence"
             xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
             xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence
             http://xmlns.jcp.org/xml/ns/persistence/persistence_2_1.xsd"
             version="2.1">

  <persistence-unit name="CadastroPU" transaction-type="JTA">
    <jta-data-source>java:jboss/datasources/loja</jta-data-source>

    <class>br.com.cadastro.entity.Produto</class>

    <properties>
      <property name="hibernate.dialect"
value="org.hibernate.dialect.SQLServerDialect"/>
      <property name="hibernate.hbm2ddl.auto" value="update"/>
      <property name="hibernate.show_sql" value="true"/>
    </properties>
  </persistence-unit>
</persistence>
```

```
package br.com.cadastro.web;

import br.com.cadastro.model.Produto;
import br.com.cadastro.ejb.ProdutoFacadeLocal;

import jakarta.ejb.EJB;
import jakarta.servlet.*;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;
import java.io.IOException;

@WebServlet(name = "ServletProdutoFC", urlPatterns = {"/ServletProdutoFC"})
public class ServletProdutoFC extends HttpServlet {

    private static final long serialVersionUID = -8010490634600155483L;

    @EJB
    private ProdutoFacadeLocal facade;

    protected void processRequest(HttpServletRequest request, HttpServletResponse
response)
            throws ServletException, IOException {

        String acao = request.getParameter("acao");
        String destino;

        if (acao == null) {
            acao = "listar";
        }

        switch (acao) {
            case "listar":
                request.setAttribute("produtos", facade.findAll());
                destino = "index.jsp";
                break;

            case "formIncluir":
                destino = "cadastro.jsp";
                break;

            case "formAlterar":
                String idParam = request.getParameter("idProduto");
                if (idParam != null && !idParam.isEmpty()) {
                    Long id = Long.parseLong(idParam);
                    Produto produtoExistente = facade.find(id);
                    request.setAttribute("produto", produtoExistente);
                    destino = "cadastro.jsp";
                } else {
                    response.sendError(HttpServletResponse.SC_BAD_REQUEST,
"Parâmetro 'idProduto' ausente ou inválido.");
                    return;
```

```
            }
            break;

        case "incluir":
            Produto novoProduto = new Produto();
            novoProduto.setNome(request.getParameter("nome"));

novoProduto.setQuantidade(Integer.parseInt(request.getParameter("quantidade")));

novoProduto.setPreco(Double.parseDouble(request.getParameter("preco")));
            facade.create(novoProduto);
            response.sendRedirect("ServletProdutoFC?acao=listar");
            return;

        case "alterar":
            String idAlterar = request.getParameter("idProduto");
            if (idAlterar != null && !idAlterar.isEmpty()) {
                Long id = Long.parseLong(idAlterar);
                Produto produtoAlterar = facade.find(id);
                produtoAlterar.setNome(request.getParameter("nome"));

produtoAlterar.setQuantidade(Integer.parseInt(request.getParameter("quantidade")));

produtoAlterar.setPreco(Double.parseDouble(request.getParameter("preco")));
                facade.edit(produtoAlterar);
                response.sendRedirect("ServletProdutoFC?acao=listar");
                return;
            } else {
                response.sendError(HttpServletResponse.SC_BAD_REQUEST,
"Parâmetro 'idProduto' ausente ou inválido.");
                return;
            }

        case "excluir":
            String idExcluir = request.getParameter("idProduto");
            if (idExcluir != null && !idExcluir.isEmpty()) {
                Long id = Long.parseLong(idExcluir);
                Produto produtoExcluir = facade.find(id);
                if (produtoExcluir != null) {
                    facade.remove(produtoExcluir);
                }
                response.sendRedirect("ServletProdutoFC?acao=listar");
                return;
            } else {
                response.sendError(HttpServletResponse.SC_BAD_REQUEST,
"Parâmetro 'idProduto' ausente ou inválido.");
                return;
            }

        default:
            destino = "index.jsp";
```

```
            break;
        }

        RequestDispatcher dispatcher = request.getRequestDispatcher(destino);
        dispatcher.forward(request, response);
    }

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        processRequest(request, response);
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse
response)
            throws ServletException, IOException {
        processRequest(request, response);
    }
}
```

```
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
         http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd"
         version="4.0">

    <welcome-file-list>
        <welcome-file>index.jsp</welcome-file>
    </welcome-file-list>

</web-app>
```

```
package br.com.cadastro.ejb;

import br.com.cadastro.model.Produto;
import jakarta.ejb.Stateless;
import jakarta.persistence.EntityManager;
import jakarta.persistence.PersistenceContext;
import java.util.List;

@Stateless
public class ProdutoFacade implements ProdutoFacadeLocal {
```



```
    @PersistenceContext(unitName = "CadastroPU")
    private EntityManager em;

    @Override
    public void create(Produto produto) {
        em.persist(produto);
    }

    @Override
    public void edit(Produto produto) {
        em.merge(produto);
    }

    @Override
    public void remove(Produto produto) {
        em.remove(em.merge(produto));
    }

    @Override
    public Produto find(Object idProduto) {
        return em.find(Produto.class, idProduto);
    }

    @Override
    public List<Produto> findAll() {
        return em.createQuery("SELECT p FROM Produto p",
Produto.class).getResultList();
    }
}
```

```
package br.com.cadastro.ejb;

import br.com.cadastro.model.Produto;
import java.util.List;
import jakarta.ejb.Local;

@Local
public interface ProdutoFacadeLocal {
```



```
    void create(Produto produto);
    void edit(Produto produto);
    void remove(Produto produto);
    Produto find(Object idProduto);
    List<Produto> findAll();
}
```

```
package br.com.cadastro.model;

import jakarta.persistence.*;

@Entity
@Table(name = "produto")
public class Produto {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY) // This is crucial
    private Long idProduto;

    private String nome;
    private Double preco;
    private Integer quantidade;

    // Constructors
    public Produto() {
    }

    public Produto(String nome, Double preco, Integer quantidade) {
        this.nome = nome;
        this.preco = preco;
        this.quantidade = quantidade;
    }

    // Getters and setters
    public Long getIdProduto() {
        return idProduto;
    }
```

```java
public void setIdProduto(Long idProduto) {
    this.idProduto = idProduto;
}

public String getNome() {
    return nome;
}

public void setNome(String nome) {
    this.nome = nome;
}

public Double getPreco() {
    return preco;
}

public void setPreco(Double preco) {
    this.preco = preco;
}

public Integer getQuantidade() {
    return quantidade;
}

public void setQuantidade(Integer quantidade) {
    this.quantidade = quantidade;
}

@Override
public String toString() {
    return "Produto{" +
            "idProduto=" + idProduto +
            ", nome='" + nome + '\'' +
            ", preco=" + preco +
            ", quantidade=" + quantidade +
            '}';
}
```

```
}
```

a) Como é organizado um projeto corporativo no NetBeans?

R: O Enterprise Application contém os subprojetos WAR e EJB, cada módulo tem seus próprios recursos e se faz o deploy com um .ear que junta tudo.

b) Qual o papel das tecnologias JPA e EJB na construção de um aplicativo para a plataforma Web no ambiente Java?

R: O JPA lida com bancos de dados e o EJB lida com as regras de sistema puxando informação do JPA quando precisa.

c) Como o NetBeans viabiliza a melhoria de produtividade ao lidar com as tecnologias JPA e EJB?

R: Ele automaticamente gera arquivos JPA a partir de um banco de dados, cria EJBs com assistentes e tem deploy e debug integrado no servidor.

d) O que são Servlets, e como o NetBeans oferece suporte à construção desse tipo de componentes em um projeto Web?

R: Os Servlets são classes Java que respondem a requisições web e o NetBeans cria Sevlets com poucos cliques, oferece templares, faz o deploy automaticamente.

e) Como é feita a comunicação entre os Serlvets e os Session Beans do pool de EJBs?

R: O Servlets chamam os métodos do bean diretamente e os EJBs são injetados usando o @EJB.

Estácio

2º Procedimento | Interface Cadastral com Servlet e JSPs

```
<%@ page contentType="text/html;charset=UTF-8" language="java" %>
<%@ page import="java.util.List" %>
<%@ page import="br.com.cadastro.model.Produto" %>
<%
    List<Produto> produtos = (List<Produto>)
request.getAttribute("produtos");
%>
<!DOCTYPE html>
<html>
<head>
    <title>Listagem de Produtos</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.cs
s" rel="stylesheet">
</head>
<body class="container mt-5">
    <h1 class="mb-4">Listagem de Produtos</h1>
    <a href="ServletProdutoFC?acao=formIncluir" class="btn btn-primary
mb-3">Novo Produto</a>
    <table class="table table-striped">
        <thead class="table-dark">
            <tr>
                <th>#</th>
                <th>Nome</th>
                <th>Quantidade</th>
                <th>Preço de Venda</th>
                <th>Opções</th>
            </tr>
        </thead>
        <tbody>
        <% if (produtos != null && !produtos.isEmpty()) {
            for (Produto p : produtos) { %>
                <tr>
                    <td><%= p.getIdProduto() %></td>
                    <td><%= p.getNome() %></td>
                    <td><%= p.getQuantidade() %></td>
                    <td>R$ <%= String.format("%.2f", p.getPreco()) %></td>
```

```
                    <td>
                        <a href="ServletProdutoFC?
acao=formAlterar&idProduto=<%= p.getIdProduto() %>" class="btn btn-sm btn-
primary">Alterar</a>
                                        <a href="ServletProdutoFC?
acao=excluir&idProduto=<%= p.getIdProduto() %>" class="btn btn-sm btn-
danger" onclick="return confirm('Tem certeza que deseja
excluir?');">Excluir</a>
                                    </td>
                </tr>
        <%   }
        } else { %>
            <tr>
                <td colspan="5" class="text-center">Nenhum produto
encontrado.</td>
            </tr>
        <% } %>
        </tbody>
    </table>
</body>
</html>
```

```
<%@ page contentType="text/html;charset=UTF-8" language="java" %>
<%@ page import="br.com.cadastro.model.Produto" %>
<%
    Produto produto = (Produto) request.getAttribute("produto");
    boolean isUpdate = (produto != null);
%>
<!DOCTYPE html>
<html>
<head>
    <title><%= isUpdate ? "Alterar Produto" : "Novo Produto" %></title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.cs
s" rel="stylesheet">
</head>
<body class="container mt-5">
    <h1><%= isUpdate ? "Alterar Produto" : "Adicionar Novo Produto" %></h1>
    <form action="ServletProdutoFC?acao=<%= isUpdate ? "alterar" : "incluir"
%>" method="post">
        <% if (isUpdate) { %>
```

```
            <input type="hidden" name="idProduto" value="<%=
produto.getIdProduto() %>">
        <% } %>
        <div class="mb-3">
            <label for="nome" class="form-label">Nome</label>
            <input type="text" id="nome" name="nome" class="form-control"
value="<%= isUpdate ? produto.getNome() : "" %>" required>
        </div>
        <div class="mb-3">
            <label for="quantidade" class="form-label">Quantidade</label>
            <input type="number" id="quantidade" name="quantidade"
class="form-control" value="<%= isUpdate ? produto.getQuantidade() : "" %>"
required>
        </div>
        <div class="mb-3">
            <label for="preco" class="form-label">Preço de Venda</label>
            <input type="number" step="0.01" id="preco" name="preco"
class="form-control" value="<%= isUpdate ? produto.getPreco() : "" %>"
required>
        </div>
        <button type="submit" class="btn btn-success">Salvar</button>
        <a href="ServletProdutoFC?acao=listar" class="btn btn-
secondary">Cancelar</a>
    </form>
</body>
</html>
```

a) Como funciona o padrão Front Controller, e como ele é implementado em um aplicativo Web Java, na arquitetura MVC?

R: O Front Controller é um servlet que recebe todas as requisições e transmite para o controller correto.

b) Quais as diferenças e semelhanças entre Servlets e JSPs?

R: Servlets lidam com a lógica enquanto os JSPs lidam com a apresentação enquanto que ambos geram HTML.

c) Qual a diferença entre um redirecionamento simples e o uso do método forward, apartir do RequestDispatcher? Para que servem parâmetros e atributos nos objetos HttpRequest?

R: Um redirecionamento simples gera uma nova requisição com um url novo enquanto que o forward() mantêm a requisição e o url antigos. Os parâmetros representam os dados do cliente enquanto que os atributos representam os dados do servidor.

3º Procedimento | Melhorando o Design da Interface

a) Como o framework Bootstrap é utilizado?

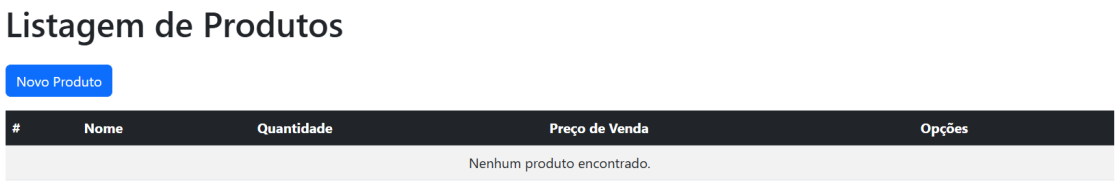
R: É um framework front-end com CSS/JS prontos.

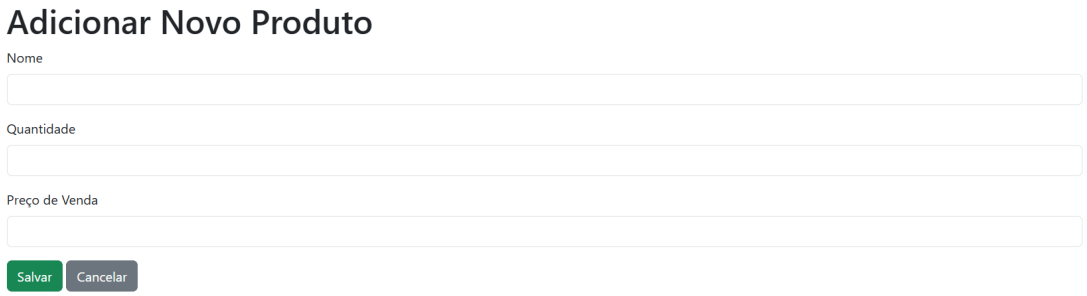
b) Por que o Bootstrap garante a independência estrutural do HTML?

R: O Bootstrap aplica estilos usando CSS.

c) Qual a relação entre o Boostrap e a responsividade da página?

R: O Boostrap tem grid que muda o layout baseado na tela.








Estácio